ARTIFICIAL INTELLIGENCE ENGINEERING

Handwritten-style Beginner Notes

CHAPTERS 1 - 3

Introduction to AI | Python Fundamentals | Intermediate Python

Prepared for: Muneeb Fayaz

Definitions • Examples • Python Code • Outputs • Practice

CONTENTS

Chapter 1 - Introduction to Artificial Intelligence (Notebook Pages 01-03)

Chapter 2 - Python Programming for AI Engineers (Notebook Pages 04-13)

Chapter 3 - Intermediate Python for AI Engineering (Notebook Pages 14-24)

Learning tip: write the definitions in a notebook, then run each code sample and change its inputs.

Note: Notebook page numbers are section labels. Actual Word/PDF print pagination may differ.

CHAPTER 1 | INTRODUCTION TO ARTIFICIAL INTELLIGENCE

Level: Beginner | Conceptual foundations

PAGE 01 - Basic Introduction

1. What is Artificial Intelligence (AI)?

Definition: Artificial Intelligence is a branch of computer science that enables machines to perform tasks that normally require human intelligence.

In simple words: AI allows computers to learn patterns, understand information, solve problems and make decisions.

Example	How AI is used
ChatGPT	Answers questions and generates text
Google Maps	Predicts traffic and suggests routes
Face Unlock	Recognizes faces
YouTube	Recommends videos

2. What is AI Engineering?

Definition: AI Engineering is the process of designing, developing, testing and deploying intelligent applications using AI technologies.

Main goal: Build useful real-world applications powered by AI. Example: a student-support chatbot.

3. How Does AI Work?

A simplified workflow for a machine-learning-based AI system is:

DATA --> TRAINING --> TRAINED MODEL --> INPUT --> PREDICTION / RESPONSE

NOTE A trained AI model can process new inputs without being retrained every time. Not every AI system follows this exact workflow.

PAGE 02 - Core AI Concepts

4. Main Branches of AI

Technology	Meaning
AI	Machines performing intelligent tasks
ML	Machine Learning - learns patterns from data
DL	Deep Learning - neural-network-based ML
Generative AI	Generates new text, images, code or other content

5. Relationship Between AI, ML and DL

Deep Learning is part of Machine Learning, and Machine Learning is part of Artificial Intelligence.

ARTIFICIAL INTELLIGENCE (AI)
  MACHINE LEARNING (ML)
    DEEP LEARNING (DL)

NOTE All deep learning is machine learning, and all machine learning is AI; not all AI is ML. Generative AI often uses DL and overlaps these categories.

6. Types of Artificial Intelligence

A. Narrow AI (ANI)

AI designed for specific tasks. Examples: voice assistants, recommendations and face recognition.

B. Artificial General Intelligence (AGI)

A proposed AI capable of handling a broad range of intellectual tasks at human-level ability or beyond. Definitions are debated.

C. Artificial Superintelligence (ASI)

A hypothetical AI surpassing human intellectual capabilities across nearly all domains.

NOTE Narrow AI exists today; AGI and ASI are not established achievements.

7. Important Terms

Term	Simple meaning
Dataset	Collection of information
Algorithm	Step-by-step method to solve a problem
Model	System that predicts, classifies or generates
Training	Process of learning patterns from data
Inference	Using a model to handle new inputs
Prompt	Input or instruction given to an AI system
API	Interface for programs to communicate
LLM	Large Language Model for language-related tasks

8. Applications of AI

- Healthcare - medical image analysis and decision support
- Education - AI tutors and personalized learning
- Banking - fraud detection
- Agriculture - crop disease detection
- Transport - route optimization
- Software development - coding and debugging assistance

9. Skills Required for an AI Engineer

1. Python programming
2. Mathematics and statistics
3. Data analysis
4. Machine learning
5. Deep learning
6. Generative AI and LLMs
7. APIs and backend development
8. Deployment and cloud basics

10. Advantages and Limitations

Advantages	Limitations
Automates repetitive tasks	May make incorrect predictions
Processes large datasets	May need strong computing power
Can improve productivity	Can carry bias from data
Scales across many users	Privacy and security concerns

Chapter 1 - Quick Revision

9. What does AI stand for?
10. What is AI Engineering?
11. What is the difference between training and inference?
12. How are AI, ML and DL related?

CHAPTER 2 | PYTHON PROGRAMMING FOR AI ENGINEERS

Level: Absolute beginner | Goal: Python foundations

PAGE 04 - Introduction to Python

1. What is Python?

Definition: Python is a high-level, interpreted, general-purpose programming language known for simple, readable syntax.

In simple words: Python helps us write instructions for computers.

2. Why is Python Used in AI?

- Easy to learn and understand
- Powerful libraries for AI and ML
- Helpful for data analysis and automation
- Large developer community
- Fast development of experiments and apps

Library	Use
NumPy	Numerical calculations
Pandas	Data analysis
Matplotlib	Charts and visualization
Scikit-learn	Machine learning
PyTorch	Deep learning

3. Our First Python Program

```
print("Hello World")
```

Output: `Hello World`

```
print("I am learning AI Engineering")
print(10 + 20)
```

Output: `I am learning AI Engineering | 30`

NOTE Text typically goes in quotation marks. Numbers can be written without quotes.

PAGE 05 - Syntax, Comments and Indentation

4. What is Syntax?

Definition: Syntax is the set of rules for writing valid code in a programming language.

```
print("Welcome to Python")
```

Output: `Welcome to Python`

Python is case-sensitive: the names "name" and "Name" refer to different variables.

```
name = "AI"
Name = "Python"
```

5. Comments

Definition: Comments help explain code and are ignored by Python during execution.

```
# This line is a comment
print("Hello AI")
```

Output: `Hello AI`

6. Indentation

Indentation means spaces at the start of a line that define blocks of code.

```
if 10 > 5:
    print("10 is greater")
```

Output: `10 is greater`

NOTE Four spaces per indentation level is the common Python convention.

PAGE 06 - Variables and Data Types

7. What is a Variable?

A variable is a name associated with a value in a program.

```
name = "Muneeb"
age = 19
course = "AI Engineering"
print(name)
print(age)
print(course)
```

Output: `Muneeb | 19 | AI Engineering`

8. Rules for Variable Names

- Start with a letter or underscore
- Cannot start with a digit
- Cannot contain spaces
- Do not use Python keywords
- Names are case-sensitive

9. Data Types

Type	Meaning	Example
int	Whole number	25
float	Decimal number	5.6
str	Text	"Python"
bool	True or False	True
list	Ordered, changeable collection	[10, 20]
dict	Key-value collection	{"age": 19}

```
age = 19
name = "Muneeb"
print(type(age))
print(type(name))
```

Output: `<class 'int'> | <class 'str'>`

NOTE type() tells you what kind of value a variable contains.

PAGE 07 - Input and Type Conversion

10. Taking User Input

input() accepts text entered by the user while a program runs.

```
name = input("Enter your name: ")
print("Hello", name)
```

Example interaction: `Enter your name: Muneeb | Hello Muneeb`

11. Type Conversion

Type conversion means changing a value from one data type to another.

Function	Result
int()	Integer
float()	Decimal number
str()	Text/string

```
age = int(input("Enter your age: "))
print(age + 5)
```

Example interaction: `Enter your age: 19 | 24`

NOTE input() always returns a string. Convert it before performing numerical calculations.

PAGE 08 - Python Operators

12. What are Operators?

Operators are symbols used to perform calculations, comparisons and logical operations.

A. Arithmetic operators

Symbol	Operation	Example
+	Addition	5 + 3 = 8
-	Subtraction	5 - 3 = 2
*****	Multiplication	5 * 3 = 15
/	Division	6 / 2 = 3.0
%	Remainder	7 % 2 = 1
******	Exponent	2 ** 3 = 8
//	Floor division	7 // 2 = 3

B. Comparison operators

Symbol	Meaning
==	Equal to
!=	Not equal to
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to

C. Logical operators

- and - true only when both conditions are true
- or - true when at least one condition is true
- not - reverses a Boolean value

```
age = 20
print(age >= 18)
print(age > 18 and age < 25)
```

Output: `True | True`

PAGE 09 - Conditional Statements

13. What are Conditional Statements?

Conditional statements let a program decide which code to run based on a condition.

Common forms: if, if-else, if-elif-else.

Example 1 - If-Else

```
age = 19
if age >= 18:
    print("You are an adult")
else:
    print("You are a minor")
```

Output: `You are an adult`

Example 2 - If-Elif-Else

```
marks = 85
if marks >= 90:
    print("Grade A")
elif marks >= 70:
    print("Grade B")
else:
    print("Grade C")
```

Output: `Grade B`

Decision flow: CONDITION? --> Yes: IF BLOCK / No: ELSE BLOCK

PAGE 10 - Loops in Python

14. What is a Loop?

A loop repeats a block of instructions.

A. For Loop

A for loop processes items in a sequence or another iterable.

```
for i in range(1, 6):
    print(i)
```

Output: `1 | 2 | 3 | 4 | 5`

NOTE range(1, 6) produces 1, 2, 3, 4 and 5. The stop value is excluded.

B. While Loop

A while loop repeats as long as a condition stays true.

```
count = 1
while count <= 3:
    print("Learning AI")
    count += 1
```

Output: `Learning AI | Learning AI | Learning AI`

Difference: for is commonly used for sequence iteration; while is helpful when repetition depends on a condition.

PAGE 11 - Lists and Dictionaries

15. Lists in Python

A list is an ordered, changeable collection. Indexing begins at 0.

```
marks = [75, 85, 90, 95]
print(marks[0])
print(marks[2])
```

Output: `75 | 90`

```
marks.append(100)
print(len(marks))
print(max(marks))
```

Output: `5 | 100`

16. Dictionaries in Python

A dictionary stores data as key-value pairs.

```
student = {
    "name": "Muneeb",
    "age": 19,
    "course": "AI"
}
print(student["name"])
print(student["course"])
```

Output: `Muneeb | AI`

NOTE Lists and dictionaries help store and organize data in AI programs.

PAGE 12 - Functions in Python

17. What is a Function?

A function is a reusable block of code that performs a particular task.

- Reduces repetitive code
- Makes programs easier to understand
- Improves code organization

Example 1 - Simple Function

```
def greet():
    print("Welcome to AI Engineering")

greet()
```

Output: `Welcome to AI Engineering`

Example 2 - Function with Parameters

```
def add(a, b):
    return a + b

result = add(10, 20)
print(result)
```

Output: `30`

NOTE def creates a function; parameters receive values; return sends a result back.

PAGE 13 - First AI-Related Python Project

Project 01 - Simple Rule-Based Chatbot

Objective: Create a chatbot that responds to a few predefined user messages.

```
message = input("You: ").lower()

if message == "hello":
    print("Bot: Hello! How can I help?")
elif message == "what is ai":
    print("Bot: AI means Artificial Intelligence.")
elif message == "bye":
    print("Bot: Goodbye!")
else:
    print("Bot: Sorry, I don't understand.")
```

Example interaction: `You: hello | Bot: Hello! How can I help?`

NOTE This is a rule-based chatbot, not a trained AI model. It teaches programming logic used later in AI applications.

Chapter 2 - Practice

13. What is the difference between print() and input()?
14. Which data type stores decimal numbers?
15. What does == mean?
16. What is the first index in a Python list?
17. What does def do?

CHAPTER 3 | INTERMEDIATE PYTHON FOR AI ENGINEERING

Level: Beginner to intermediate | Data-ready Python

PAGE 14 - Strings in Python

1. What is a String?

A string is a sequence of characters enclosed in single or double quotation marks.

```
name = "Muneeb"
course = 'AI Engineering'
print(name)
print(course)
```

Output: `Muneeb | AI Engineering`

2. String Indexing

```
text = "PYTHON"
print(text[0])
print(text[2])
print(text[-1])
```

Output: `P | T | N`

Index positions for P Y T H O N are 0 1 2 3 4 5. Negative indexing counts from the right.

3. String Slicing

Slicing selects characters with string[start:stop], excluding stop.

```
text = "PYTHON"
print(text[0:3])
print(text[2:5])
```

Output: `PYT | THO`

4. Common String Methods

Method	Purpose
upper()	Uppercase letters
lower()	Lowercase letters
strip()	Remove outer spaces
replace()	Substitute text
split()	Split into a list

```
message = " Hello AI "
print(message.strip())
print(message.upper())
```

Output: `Hello AI | HELLO AI`

NOTE String operations help clean text used in natural language processing (NLP).

PAGE 15 - Tuples and Sets

5. What is a Tuple?

A tuple is an ordered, immutable collection. Immutable means its elements cannot be modified in place.

```
numbers = (10, 20, 30)
print(numbers[0])
print(numbers[2])
```

Output: `10 | 30`

6. What is a Set?

A set is an unordered collection of unique elements.

```
numbers = {10, 20, 20, 30}
print(len(numbers))
```

Output: `3`

7. List vs Tuple vs Set

Feature	List	Tuple	Set
Ordered	Yes	Yes	No
Duplicates	Yes	Yes	No
Changeable	Yes	No	Yes
Syntax	[ ]	( )	{ }

NOTE Sets are handy for finding unique labels; tuples are useful for fixed groups of values.

PAGE 16 - List Comprehension

8. What is List Comprehension?

List comprehension is a concise way to create a list using an iterable and an optional condition.

Normal Loop

```
squares = []
for x in range(1, 5):
    squares.append(x * x)
print(squares)
```

Output: `[1, 4, 9, 16]`

Same Result with List Comprehension

```
squares = [x * x for x in range(1, 5)]
print(squares)
```

Output: `[1, 4, 9, 16]`

Comprehension with a Condition

```
numbers = [1, 2, 3, 4, 5, 6]
even = [x for x in numbers if x % 2 == 0]
print(even)
```

Output: `[2, 4, 6]`

NOTE Data cleaning and filtering tasks often use list comprehensions.

PAGE 17 - Advanced Functions

9. Parameters and Arguments

Parameters are names listed in a function definition; arguments are the values passed when calling it.

```
def greet(name):
    print("Hello", name)

greet("Muneeb")
```

Output: `Hello Muneeb`

10. Default Arguments

```
def greet(name="Student"):
    print("Welcome", name)

greet()
greet("Muneeb")
```

Output: `Welcome Student | Welcome Muneeb`

11. Lambda Functions

A lambda expression creates a small anonymous function.

```
square = lambda x: x * x
print(square(5))
```

Output: `25`

12. map() Function

map() applies a function to each item of an iterable.

```
numbers = [1, 2, 3]
result = list(map(lambda x: x * 2, numbers))
print(result)
```

Output: `[2, 4, 6]`

NOTE Reusable functions help organize AI preprocessing and application code.

PAGE 18 - Modules and Packages

13. What is a Module?

A module is a Python file that provides reusable functions, classes or variables.

Math Module

```
import math
print(math.sqrt(25))
print(math.factorial(5))
```

Output: `5.0 | 120`

Random Module

```
import random
number = random.randint(1, 10)
print(number)
```

Possible output (random result varies): `7`

14. What is a Package?

Packages help organize related Python modules. Popular third-party Python tools include NumPy, Pandas, Scikit-learn and PyTorch.

15. Installing Packages with pip

```
python -m pip install numpy
python -m pip install pandas
```

NOTE Run pip commands in a system terminal, not within the regular Python prompt.

PAGE 19 – File Handling

16. What is File Handling?

File handling means creating, reading, writing and updating files using Python.

17. Common File Modes

Mode	Purpose
r	Read an existing file
w	Write, replacing existing content
a	Append new content
x	Create only if not already present

Write to a File

```
with open("notes.txt", "w") as file:
    file.write("I am learning AI")
```

Result: `A notes.txt file containing: I am learning AI`

Read from a File

```
with open("notes.txt", "r") as file:
    content = file.read()
print(content)
```

Output: `I am learning AI`

Append to a File

```
with open("notes.txt", "a") as file:
    file.write("\nPython is useful")
```

Updated file contents: `I am learning AI | Python is useful`

NOTE The with statement automatically closes an opened file.

PAGE 20 - Exception Handling

18. What is Exception Handling?

Exception handling lets a program respond to runtime errors instead of unexpectedly stopping.

Example: Division by Zero

```
try:
    a = 10
    b = 0
    result = a / b
    print(result)
except ZeroDivisionError:
    print("Cannot divide by zero")
```

Output: `Cannot divide by zero`

19. try / except / else / finally

```
try:
    number = int("25")
except ValueError:
    print("Invalid number")
else:
    print("Conversion successful")
finally:
    print("Program completed")
```

Output: `Conversion successful | Program completed`

- try - code that could raise an error
- except - handles a particular error
- else - runs if no exception occurs
- finally - runs whether an error occurs or not

NOTE Exception handling helps AI apps cope with invalid input, missing files and failed API calls.

PAGE 21 - Object-Oriented Programming (OOP)

20. What is OOP?

Object-Oriented Programming organizes software into classes and objects.

21. What is a Class?

A class is a blueprint used to create objects.

22. What is an Object?

An object is an instance of a class and holds data (attributes) plus behavior (methods).

Example concept: Student class --> Student("Muneeb", 19), Student("Ali", 20).

```
class Student:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def display(self):
        print(self.name, self.age)

s1 = Student("Muneeb", 19)
s1.display()
```

Output: `Muneeb 19`

- class - defines a class
- __init__() - initializes object attributes
- self - current object
- display() - a class method
- s1 - one object of the class

PAGE 22 - Inheritance and OOP Principles

23. What is Inheritance?

Inheritance lets a class reuse and extend the attributes and methods of another class.

```
class Animal:
    def speak(self):
        print("Animal makes a sound")

class Dog(Animal):
    def bark(self):
        print("Dog is barking")

d = Dog()
d.speak()
d.bark()
```

Output: `Animal makes a sound | Dog is barking`

24. Four Main OOP Principles

Principle	Meaning
Encapsulation	Combines data and behavior with controlled access
Inheritance	Reuses functionality from another class
Polymorphism	Same interface, different behavior
Abstraction	Hides unnecessary implementation details

NOTE OOP helps organize larger AI programs into data loaders, models, APIs and prediction components.

PAGE 23 - JSON and CSV Data

25. What is JSON?

JSON (JavaScript Object Notation) is a lightweight structured data format common in APIs.

```
import json
student = {"name": "Muneeb", "course": "AI"}
data = json.dumps(student)
print(data)
```

Output: `{"name": "Muneeb", "course": "AI"}`

26. What is CSV?

CSV (Comma-Separated Values) stores tables as rows of text separated by commas.

```
Name,Marks
Muneeb,90
Ali,85
Sara,95
```

Sample data above is the contents of a file named students.csv (not Python code).

Reading CSV

```
import csv
with open("students.csv", "r", newline="") as file:
    reader = csv.reader(file)
    for row in reader:
        print(row)
```

Output: `['Name', 'Marks']` | `['Muneeb', '90']` | `['Ali', '85']` | `['Sara', '95']`

NOTE CSV values read this way are strings. JSON is common for AI services; CSV is common for datasets.

PAGE 24 - Python Mini Project

Project 02 - Simple Text Sentiment Checker

Objective: Classify a short message as positive, negative or unknown by looking for predefined words.

```
def check_sentiment(text):
    positive = {"good", "happy", "great", "love"}
    negative = {"bad", "sad", "hate", "angry"}

    words = text.lower().split()
    pos = sum(word in positive for word in words)
    neg = sum(word in negative for word in words)

    if pos > neg:
        return "Positive"
    elif neg > pos:
        return "Negative"
    else:
        return "Neutral / Unknown"

message = input("Enter a sentence: ")
result = check_sentiment(message)
print("Sentiment:", result)
```

Example interaction: `Enter a sentence: I love Python` | `Sentiment: Positive`

Another example: "I feel sad" --> Negative.

NOTE This is a rule-based checker, not a trained ML model. It misses context, punctuation and many words.

Chapter 3 - Revision Questions

18. What is a tuple and how is it different from a list?
19. How does list comprehension work?
20. What is the role of except?
21. What is the difference between a class and an object?
22. What formats do JSON and CSV store?

Next: Chapter 4 - NumPy for AI Engineering

After practicing these chapters, learn NumPy arrays, dimensions, shapes, indexing, slicing, broadcasting and numerical operations.